



数据结构与算法分析-1



人工智能与自动化学院

韩守东

shoudonghan@hust.edu.cn

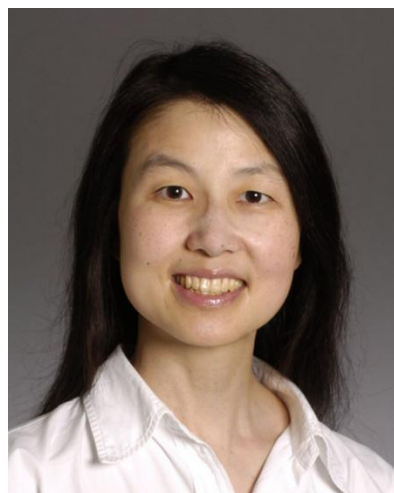
13720301586



1. 绪论

- ▶ 前言（计算思维）
- ▶ 1.1 数据结构的概念及术语
- ▶ 1.2 数据结构的逻辑结构和物理（存储）结构
- ▶ 1.3 C语言回顾
- ▶ 1.4 抽象数据类型
- ▶ 1.5 算法和算法分析

计算思维（周以真）



- CMU教授
- 现哥伦比亚大学常务副校长
- 美国艺术与科学院院士
- 美国国家发明家科学院院士
- 美国国家工程院院士

Viewpoint | Jeannette M. Wing

<http://www.cs.cmu.edu/afs/cs/usr/wing/www/publications/Wing06.pdf>

Computational Thinking

It represents a universally applicable attitude and skill set everyone, not just computer scientists, would be eager to learn and use.



Computational thinking builds on the power and limits of computing processes, whether they are executed by a human or by a machine. Computational methods and models give us the courage to solve problems and design systems that no one of us would be capable of tackling alone. Computational think-

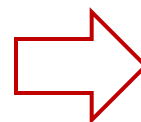
cisely. Stating the difficulty of a problem accounts for the underlying power of the machine—the computing device that will run the solution. We must consider the machine's instruction set, its resource constraints, and its operating environment.

In solving a problem efficiently, we might first ask whether an approximate solution is good enough, whether we can use randomization to our advantage, and whether false positives or false negatives are allowed. Computational thinking is ref

阅读
写作
算术



求解问题
设计系统
理解行为



抽象问题并描述

信息处理
流程



计算机
概念



计算思维-生活中的例子



看菜谱做家宴

1、分析问题

根据口味，做哪几道菜？
四菜一汤：螃蟹、牛肉、茄子、鱿鱼和鸡汤

2、模式识别寻规律

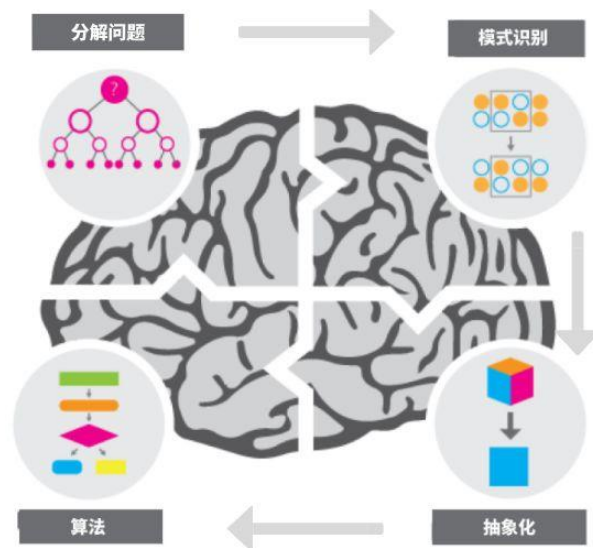
每个菜的做法，分别需要什么共性的食材、炊具和工序

3、问题抽象

几道菜不能串味，不能出现菜凉，安排做菜顺序，抽象为排序

4、执行操作

切菜、配菜，按菜谱制作流程执行，切鸡肉、姜-炖鸡汤-切蒜、葱、腌制牛肉-切螃蟹等



计算思维类型

模块化：很多菜谱都有“勾芡”这个步骤，代表着一个操作序列——取一些淀粉，加点水，搅拌均匀，在适当时候倒入菜中。（找共性，建模板）

回溯：菜做好后发现淡了或者不够鲜，回顾做菜放调料的过程。（梯度回传）

查找：找菜谱时，直接跳到对应类型的菜单。

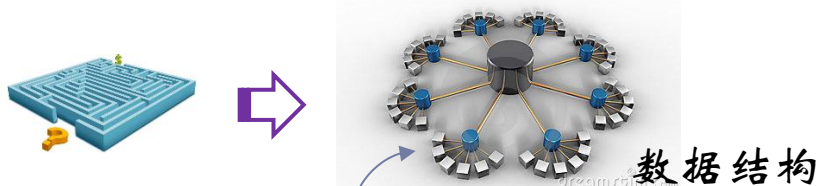
并发：在炖鸡汤时，同时准备其他菜肴的配菜等。

抽象、递归、分解、启发式推理、搜索

课程学习中建立计算思维

数据

关系



算法逻辑

高内聚
低耦合

递归

循环

逻辑判断



应用数据结构设计程序的思维过程包括：
理解问题并抽象模型、设计数据结构和算法、程序设计和编程、测试解决

解决问题的思路

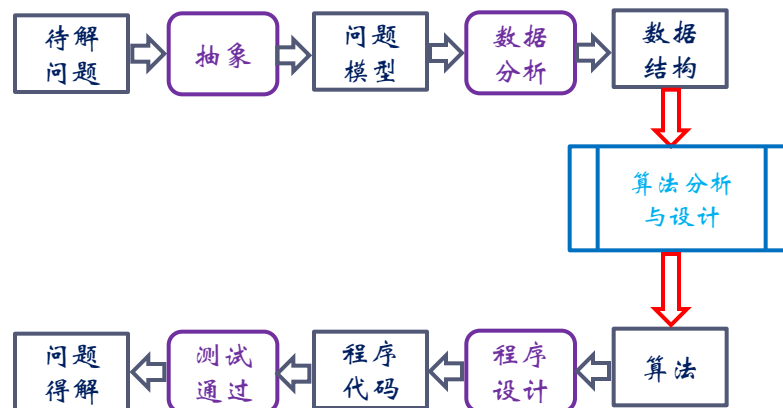
实际业务的抽象描述

计算思维

站在机器实现角度描述解决过程（数据结构知识，计算资源约束）

编程语言实现

具体语言实现



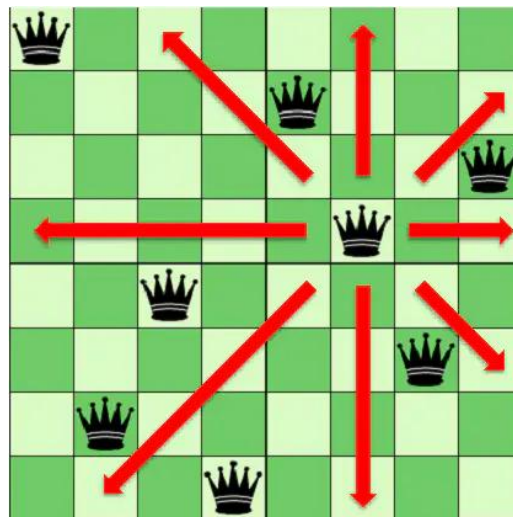


计算思维例子-8皇后问题

- **问题：** 在 8×8 格的国际象棋上摆放八个皇后，使其不能互相攻击，即任意两个皇后都不能处于同一行、同一列或同一斜线上，问一共有多少种摆法。

问题核心约束： 是否有两皇后产生攻击

- 暴力枚举：对每个组合判断是否有攻击，八重循环执行。
- 进一步优化：依次按列组合，判断第 i 列的皇后是否会被前面的 $i-1$ 列皇后攻击到。
- 更优化：回溯。选择了第一个皇后的位置之后，第二个皇后就不能选择其可辐射到的位置，第三个皇后不能选择1、2皇后辐射到的位置，依次进行，若第 i 个皇后在此约束下，无位置可选，我们就应该回溯到第 $i-1$ 个皇后重新选择其位置。



计算思维例子-8皇后问题

▶ 数据分析（数据结构）

- 8个皇后的位置表示- 缺省已知列，则只需表示行
- 因此可以用一维数组q表示组合



▶ 暴力枚举（算法）

每一列均有1个皇后，则行号形成一个组合，判断每个组合是否安全

▶ 程序设计

- 8重循环，每列的皇后取1个行位置，形成q(1),...,q(8)组合
- 在每个循环中判断该组合是否符合安全要求，若符合输出，否则取下一个组合
- 语言实现（例如C语言）

```
int q[9], num = 0;
for (q[1] = 1; q[1] <= 8; q[1]++){
    for (q[2] = 1; q[2] <= 8; q[2]++){
        for (q[3] = 1; q[3] <= 8; q[3]++){
            for (q[4] = 1; q[4] <= 8; q[4]++){
                for (q[5] = 1; q[5] <= 8; q[5]++){
                    for (q[6] = 1; q[6] <= 8; q[6]++){
                        for (q[7] = 1; q[7] <= 8; q[7]++){
                            for (q[8] = 1; q[8] <= 8; q[8]++){
                                // 判断是否安全
                                if (isSafe(q)) {
                                    num++;
                                    // 输出当前方案
                                    cout << num << ": ";
                                    for (int i = 1; i <= 8; i++) {
                                        cout << q[i] << " ";
                                    }
                                    cout << endl;
                                }
                            }
                        }
                    }
                }
            }
        }
    }
}
```

人工智能与算法

▶ 人工智能成为热点的三大原因



受限的场景：辅助驾驶，人脸识别，语音识别



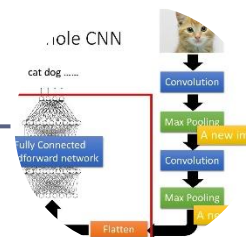
大数据

(训练样本)



计算能力

(GPU, 集群)



算法

(深度学习)

学好数据结构与算法分析的作用

► 开发工作

转 数据结构经典算法面试题

2017年04月15日 11:30:32 阅读量: 14146 更多

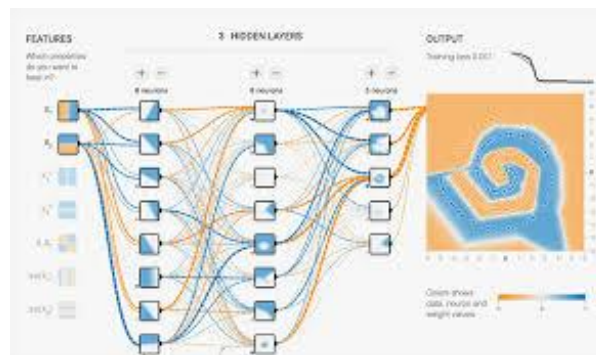


转自: http://hi.baidu.com/geogre_jsj/blog/item/e4b98fd2aab5aa3611df9b92.html

由于这些题, 实在太火了。所以, 应广大网友建议要求, 在此把之前已整理公布的前80题,

现在, 一次性分享出来。此也算是前80题第一次集体亮相。

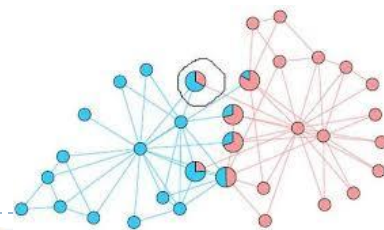
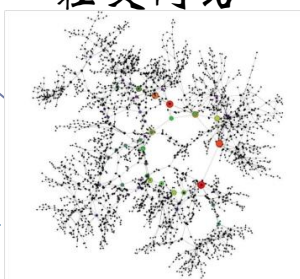
► 研究工作



Deep Learning



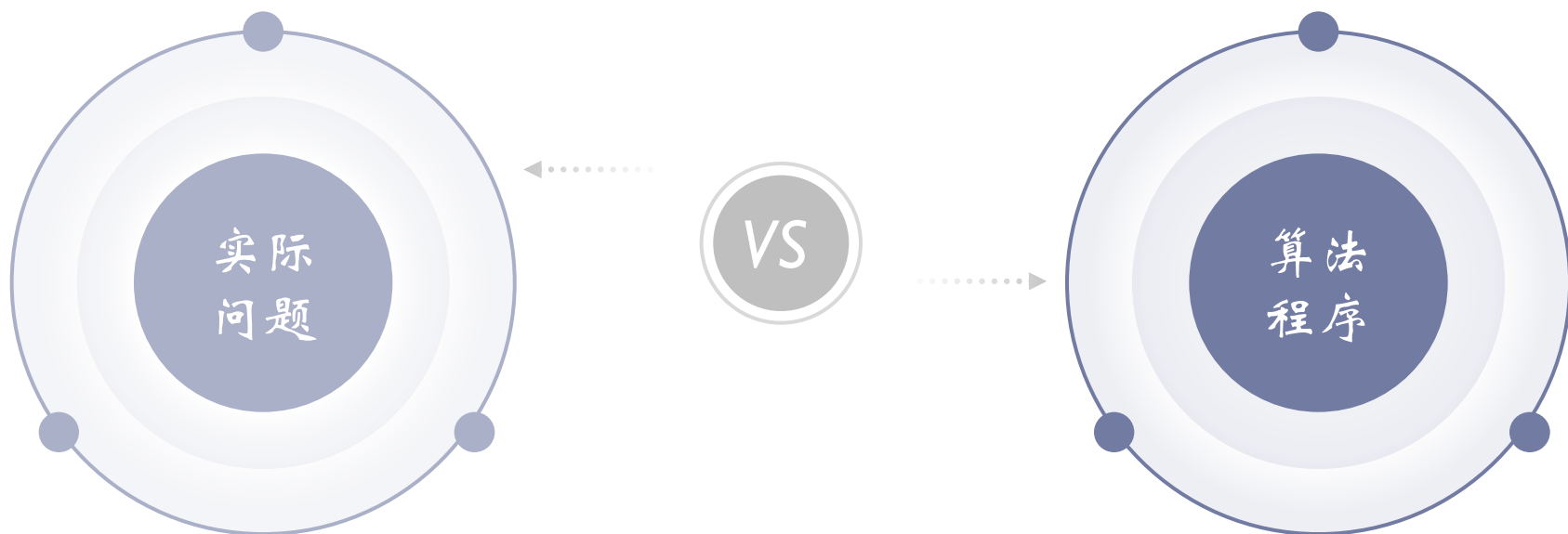
社交网络





小结

- ▶ 数字世界实现的基础：将真实世界的**数据组织起来**，形成相互关联的**数据结构**，并根据逻辑形成**算法**



- 问题抽象
- 数据描述
- 数据组织
- 逻辑流程

- 数据结构设计
- 基于算法逻辑（递归、分解、回溯等）设计算法
- 编程实现

1.1 数据结构的概念及术语

- ▶ 典型数据结构的应用例子
- ▶ 数据结构的基本概念



数据结构

=



数据

+



关系

- 将数据组织起来，并表征数据间的关系
- 真实世界各种数据是否可以归类为某些类型？
- 若是？我们是否可以定义一些抽象的数据结构？

例1：书目自动检索系统



书目卡片

登 录 号：
分 类 号：
书 名：
作 者：
出版单位：
出版时间：

线性表

书目文件

001	高等数学	樊映川	S01
002	理论力学	罗远祥	L01
003	高等数学	华罗庚	S01
004	线性代数	栾汝书	S02
.....			

按书名

高等数学	001,003
理论力学	002
线性代数	004
.....	

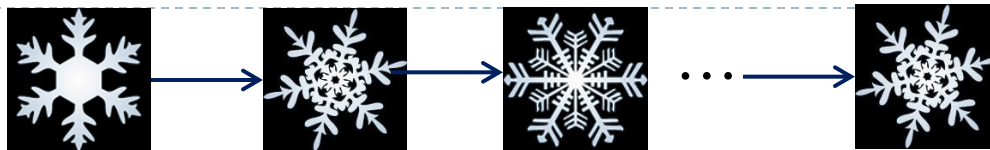
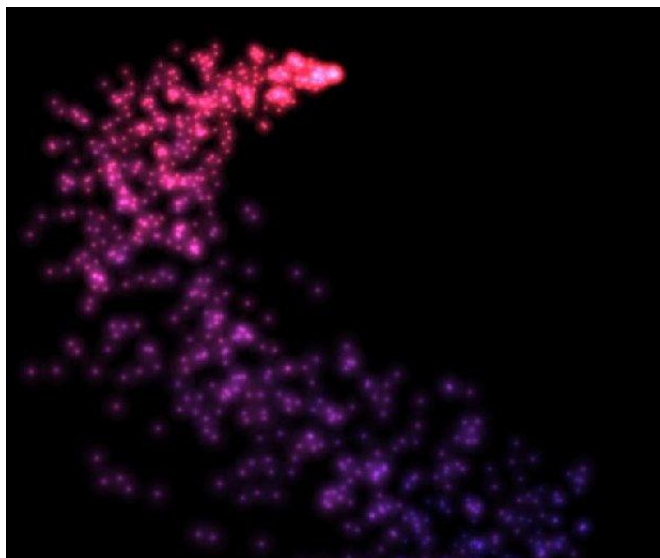
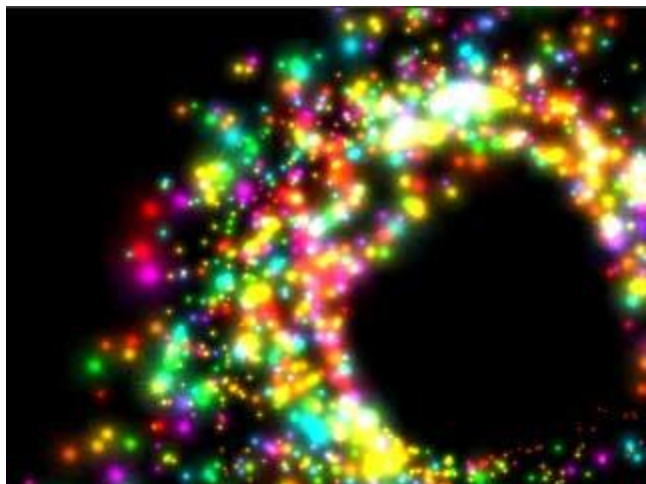
按作者

樊映川	001
罗远祥	002
华罗庚	003
栾汝书	004
.....	

按分类号

S	S01	001,003
	S02	004
L	L01	002
...		

例2：游戏中的粒子系统



```
//-----  
// Desc: 雪花粒子结构体的定义  
//-----
```

```
struct SNOWPARTICLE
```

```
{
```

```
    float x, y, z;    //坐标位置
```

```
    float RotationY;    //雪花绕自身Y轴旋转角度
```

```
    float RotationX;    //雪花绕自身X轴旋转角度
```

```
    float FallSpeed;    //雪花下降速度
```

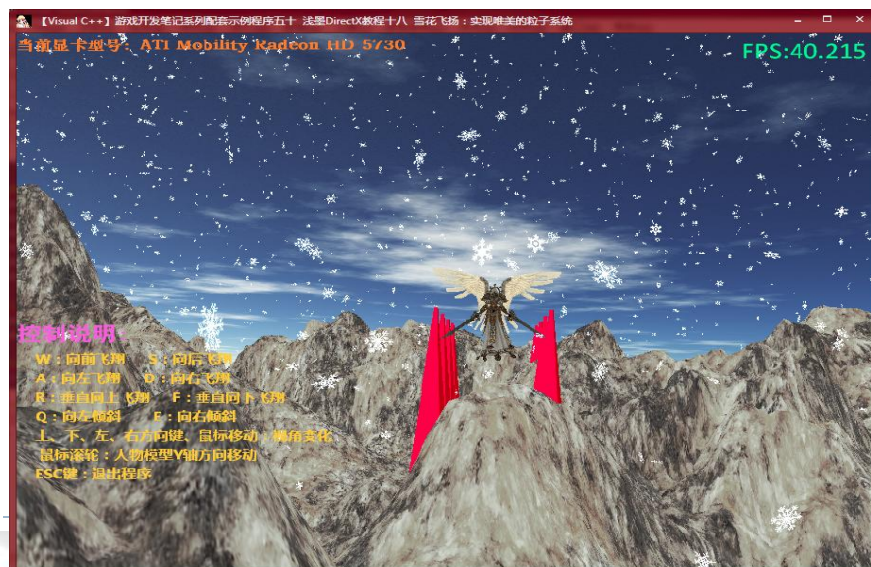
```
    float RotationSpeed;    //雪花旋转速度
```

```
    int TextureIndex;    //纹理索引数
```

```
}
```

线性表

➤ 使用数组或链表
管理所有的粒子





例3：人机对弈问题-AlphaGo



AlphaGo：2016年战胜李世石，2017年战胜柯洁。

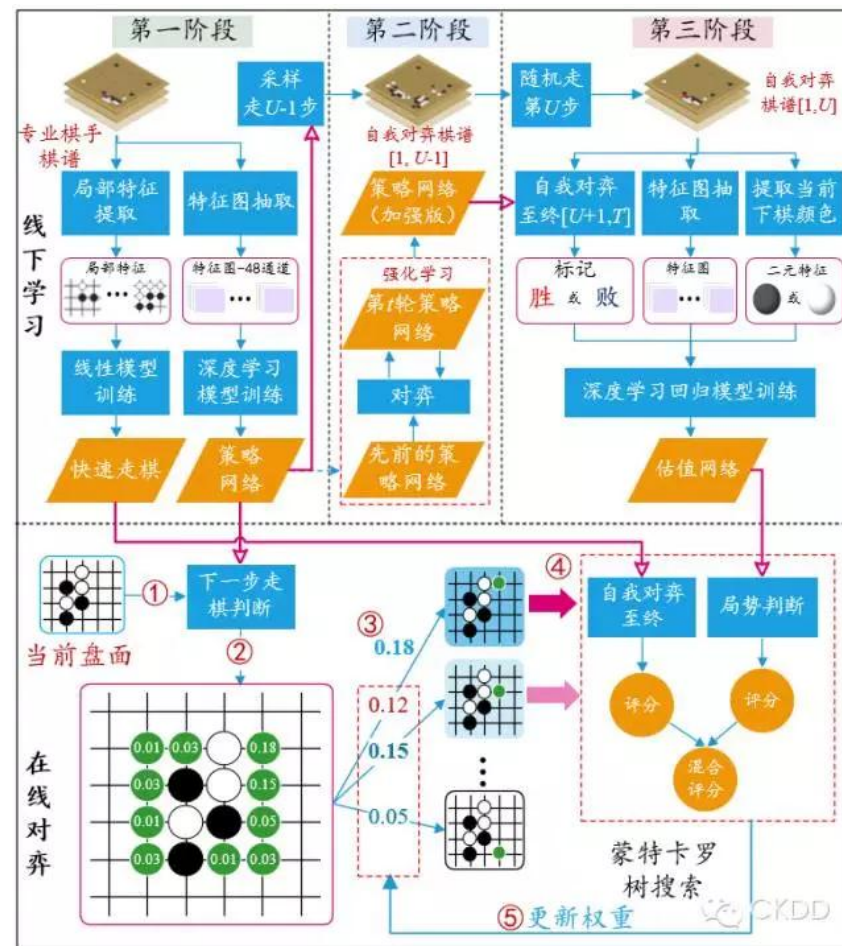
1.0版：人类棋局

2.0版：自我对弈产生棋局

核心：强化学习与蒙特卡罗树搜索



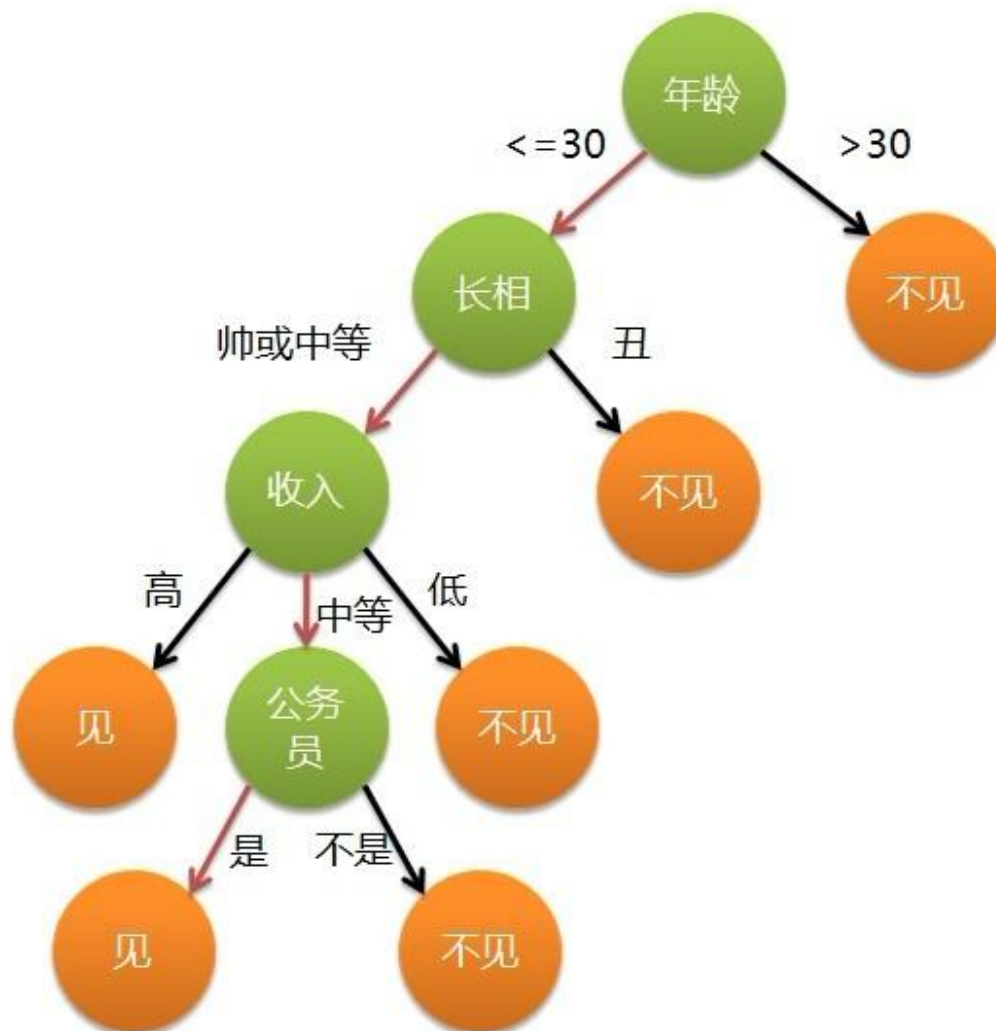
黄士杰



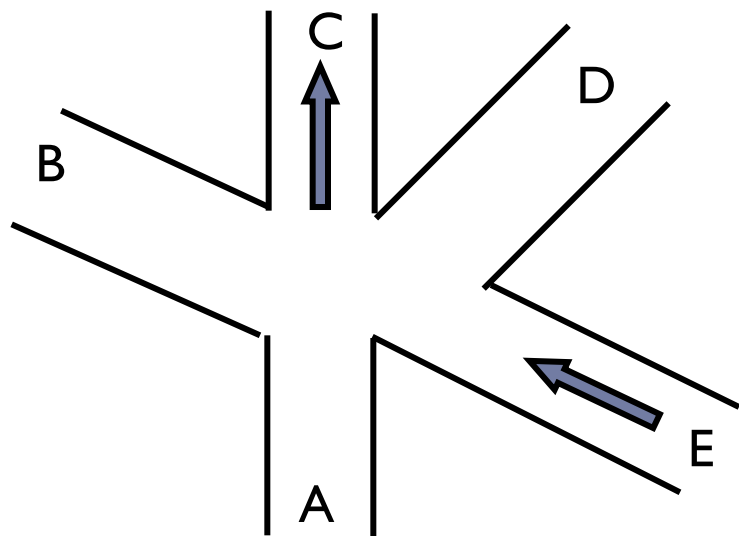
来自CKDD,郑宇和张钧波博士



例4：决策树分类

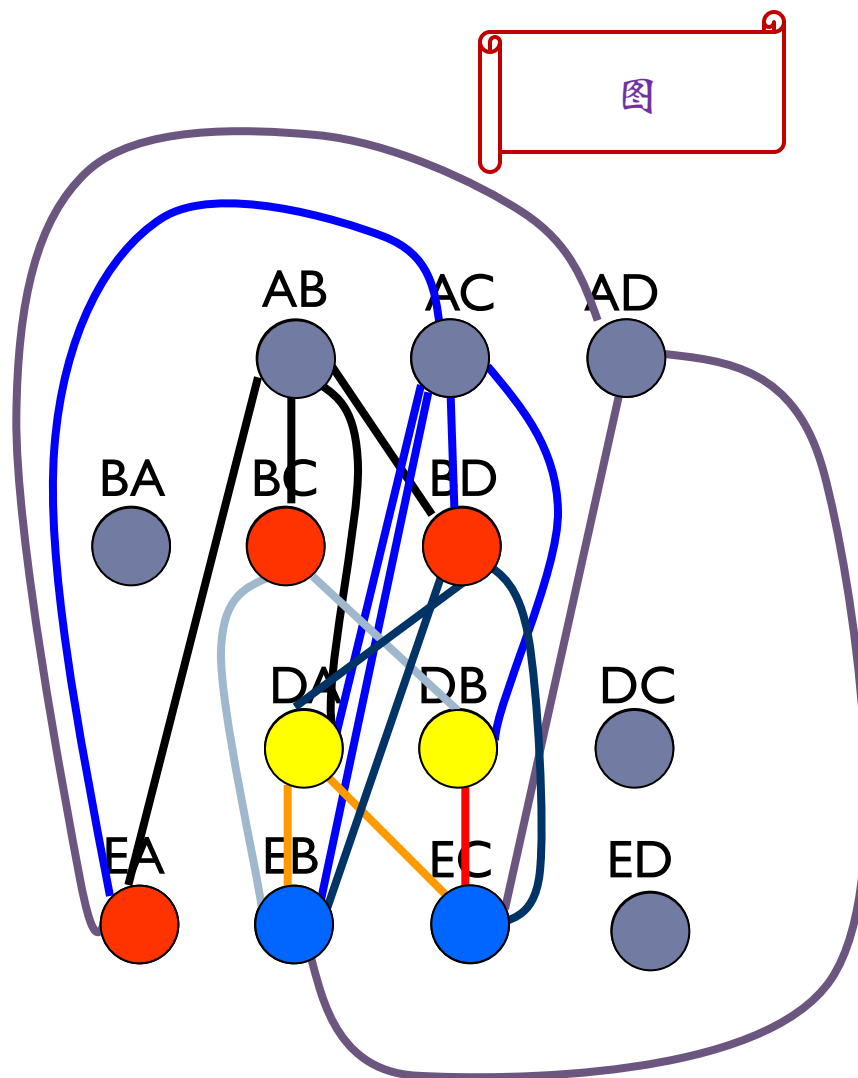


例5：交通路口信号灯问题



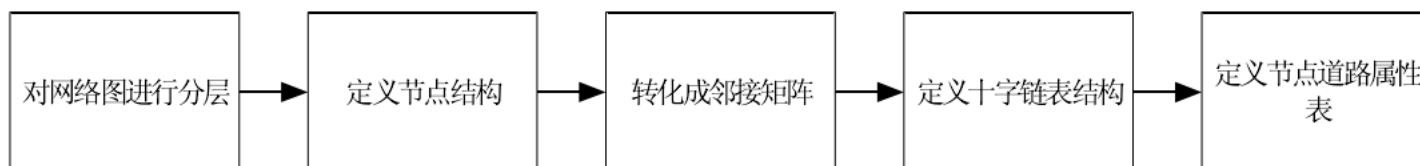
染色问题：

- 顶点为通路
- 顶点相连表示通路互斥
- 每个顶点染个颜色，相连的顶点颜色不同，且颜色种类尽可能少

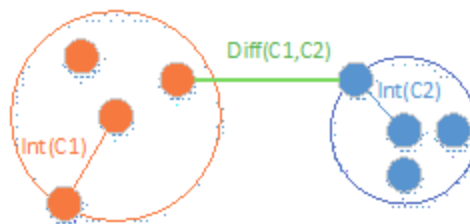




例6：个人导航



例7：图像分割



各类软件需用到数据结构

操作系统:



工业软件:



人工智能平台:



1.7万亿参数，超越谷歌Switch Transformer



2000亿参数



从例子到数据结构

▶ 程序解决实际问题的过程



例如：梁架结构应力的线性方程；人口预测中的微分方程

- ▶ 前述例子非数值类问题，无法用方程表达数学模型
- ▶ 需寻求数据结构（表、树和图）等表达问题
- ▶ **数据结构**是一门研究（非数值计算的）程序设计问题中所出现的**计算机操作对象**以及它们之间的**关系**和**操作**的学科。



数据结构的基本概念

▶ 数据

信息的载体，是描述客观事物的数、字符、以及所有能输入到计算机中，被计算机程序识别和处理的符号（数值、字符等）的集合。

▶ 数据元素（数据成员）

数据的基本单位。在不同的条件下，数据元素又可称为元素、结点、顶点、记录等。

▶ 数据对象

具有相同性质的数据元素（数据成员）的集合。例如：

整数数据对象： $N = \{0, \pm 1, \pm 2, \dots\}$ ；学生数据对象

▶ 数据结构(data structure)

数据元素和数据元素关系的集合，可表达为：

$$\text{Data_Structure} = \{D, R\}$$

D为数据元素集合，R为元素间关系的有限集合。



数据结构的基本类型

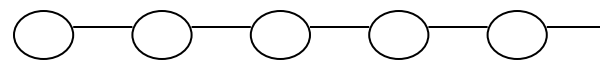
- ▶ 根据数据元素间关系的基本特性，有四种基本数据结构

- ▶ 集合——也称散列结构



数据元素间除“同属于一个集合”外，无其它关系

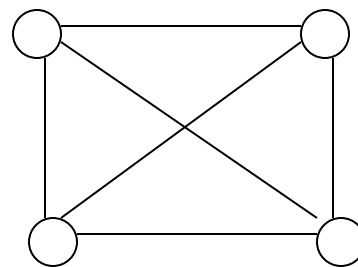
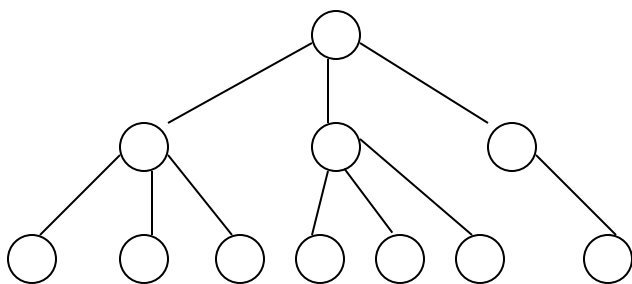
- ▶ 线性结构——1对1，如线性表、栈、队列



- ▶ 树形结构——1对多

- ▶ 图状结构——多对多

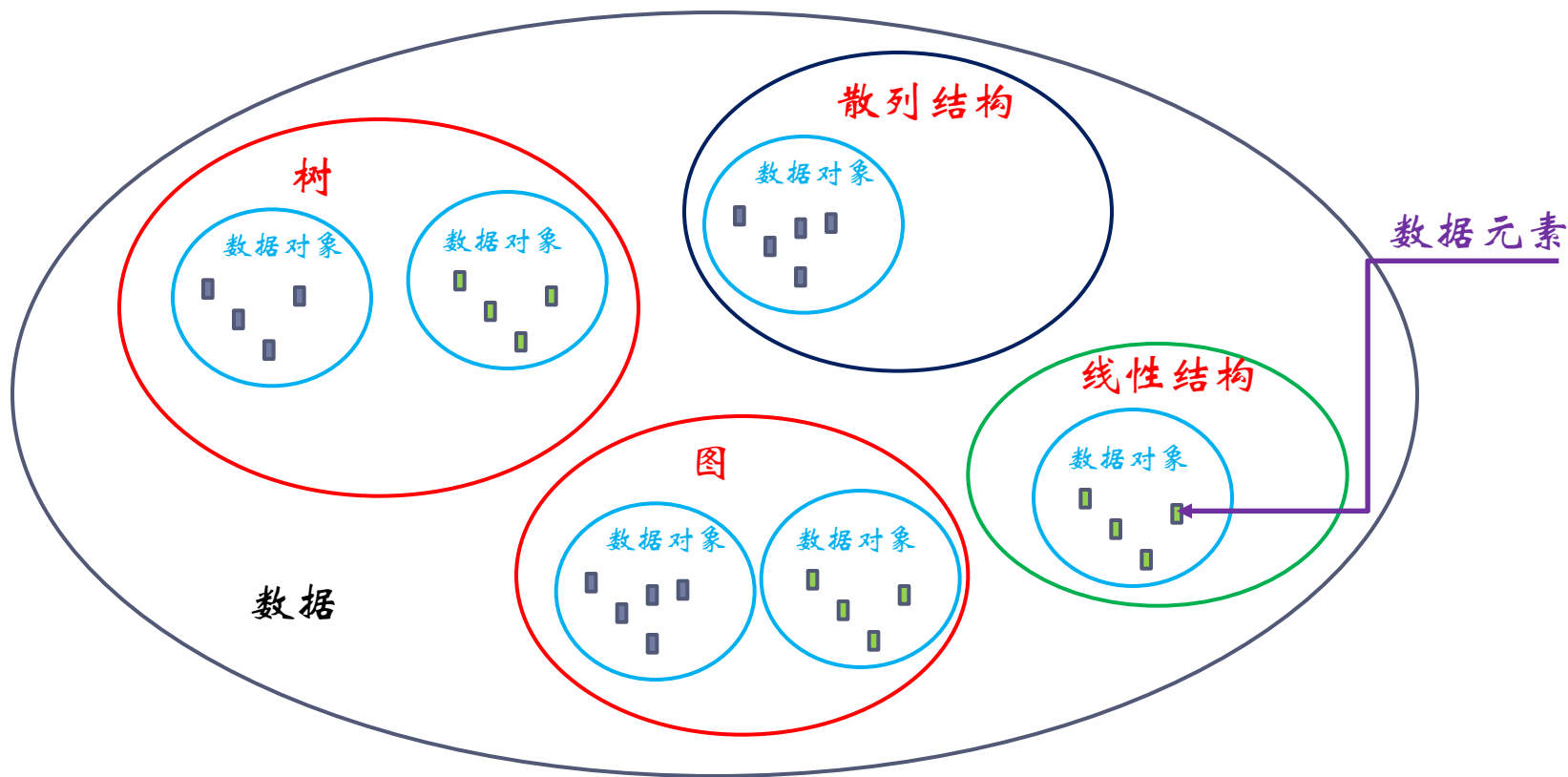
非线性结构





小结

- ▶ 广义学科角度：数据结构为数据+关系+操作
- ▶ 狭义角度：数据+关系





1.2 数据结构的逻辑结构和物理结构

数据结构的表示涉及两方面：

- ▶ 数据的逻辑结构-----从用户视图看，是面向问题的。
- ▶ 数据的物理结构-----从具体实现视图看，是面向计算机的。

例：学生表：逻辑结构-----线性表

物理结构-----数组，链表



1.2 数据结构的逻辑结构和存储结构

▶ 数据结构中的两个层次：

- ▶ 逻辑结构：是对数据成员之间的逻辑关系的描述，它可以用一个数据成员的集合和定义在此集合上的若干关系来表示；
- ▶ 物理（存储）结构：是逻辑结构在计算机中的表示和实现，故又称“存储结构”。

▶ 数据存储结构的表示：并非直接用机器内存表示。

- ▶ 高级语言—虚拟机—虚拟存储器

▶ 数据的逻辑结构与存储结构密切相关

- ▶ 算法设计 → 逻辑结构，应该与语言无关的，独立于计算机
- ▶ 算法实现 → 存储结构，与语言相关，与计算机相关。



根据问题建立逻辑结构

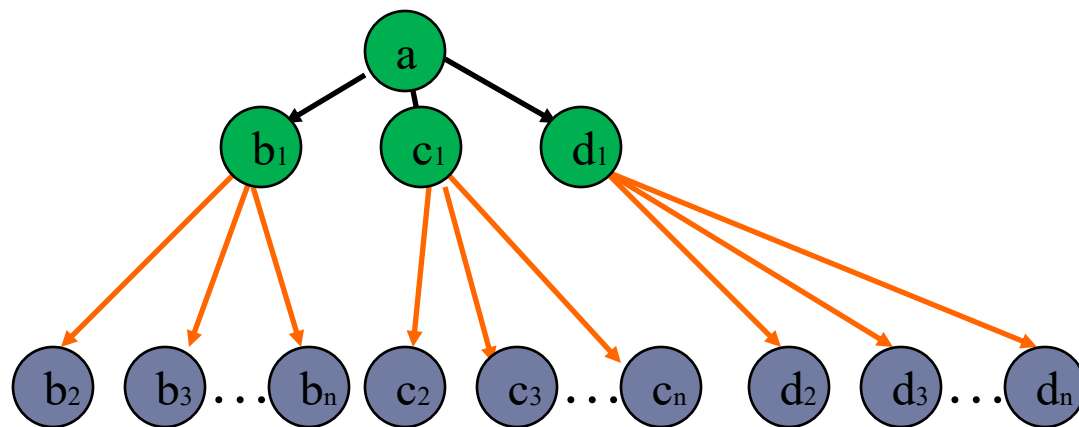
▶ 例： Class = (D, S)

▶ 数据集合： $D = \{ a, b_1, \dots, b_n, c_1, \dots, c_n, d_1, \dots, d_n \}$

▶ 关系集合： $S = \{ R_1, R_2 \}$

$R_1 = \{ \langle a, b_1 \rangle, \langle a, c_1 \rangle, \langle a, d_1 \rangle \}$; //班长----组长

$R_2 = \{ \langle b_1, b_j \rangle, \langle c_1, c_j \rangle, \langle d_1, d_j \rangle \mid j = 2, 3, \dots, n \}$; //组长----组员



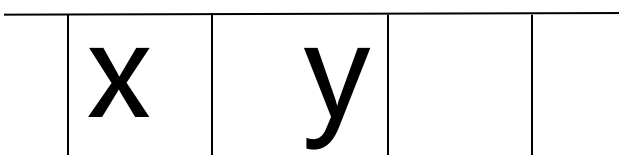
班级Class的逻辑结构的图示



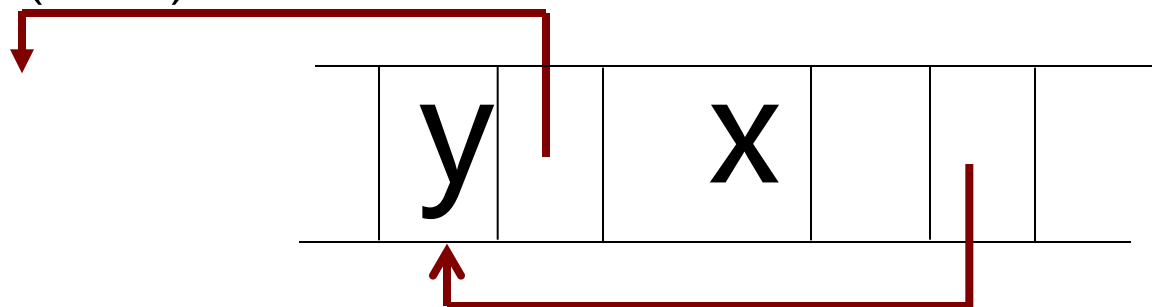


根据逻辑结构建立存储结构映像

- ▶ **存储结构**是逻辑结构在存储器中的映象。
 - ▶ **数据元素的映象**：任何的数据元素在计算机中最终都是转化成一个二进制的位串。
 - ▶ **关系的映象**：顺序和链式映像（存储结构）
 - ▶ 顺序映像（存储结构）：借助元素在存储器中的**相对位置**表示后继关系



- ▶ 链式映像（存储结构）：借助元素在存储器中的附加信息(指针)表示后继关系





顺序存储

- ▶ 借助元素在存储器中的相对位置来表示数据元素之间的逻辑关系

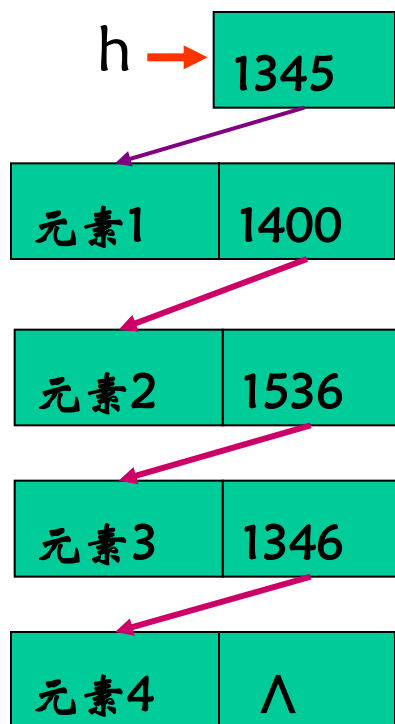
存储地址	存储内容
L_0	元素1
L_0+m	元素2
.....	
$L_0+(i-1)*m$	元素i
.....	
$L_0+(n-1)*m$	元素n

$$\text{Loc}(\text{元素}i) = L_0 + (i-1)*m$$



链式存储

- ▶ 借助指示数据元素存储地址的指针表示数据元素之间的逻辑关系。



存储地址	存储内容	指针
1345	元素1	1400
1346	元素4	∧
.....		
1400	元素2	1536
.....		
1536	元素3	1346



小结：数据结构两面性

- ▶ **Data Structure**=数据的逻辑结构+存储结构
 - 逻辑结构仅表达抽象关系，和编程实现无关
 - 存储结构通过实际的存储方式建立数据元素间的关系，和编程实现有关



1.3 C语言回顾

- ▶ C语言的成份
- ▶ C语言语句
- ▶ C语言变量
- ▶ C语言的数据类型
- ▶ 函数
- ▶ 程序结构
- ▶ 结构和联合
- ▶ 指针



C语言的成份

► C语言的成份:

► 语句

► 定义 ➡

- 变量定义
- 函数定义

► 申明 ➡

- 变量申明
- 数据类型申明
- 函数原型申明
- 编译指示申明

► 注释



C语言语句

▶ C语言的语句用“;”标记。分为以下几种：

- ▶ 赋值语句/表达式语句
- ▶ 函数调用(子程序)
- ▶ 流程控制语句
- ▶ 空语句



■ 复合语句
(用一对{}括起来)



C语言变量的定义与申明

▶ 变量的定义

存储类型 数据类型 变量名 = 初始值;

```
int x;
```

```
int x = 3;
```

```
static int x = 3;
```

▶ 变量的作用域：变量只在定义它的(函数)范围内起作用

▶ 在所有函数体之外定义的变量称全局变量

▶ 变量的申明

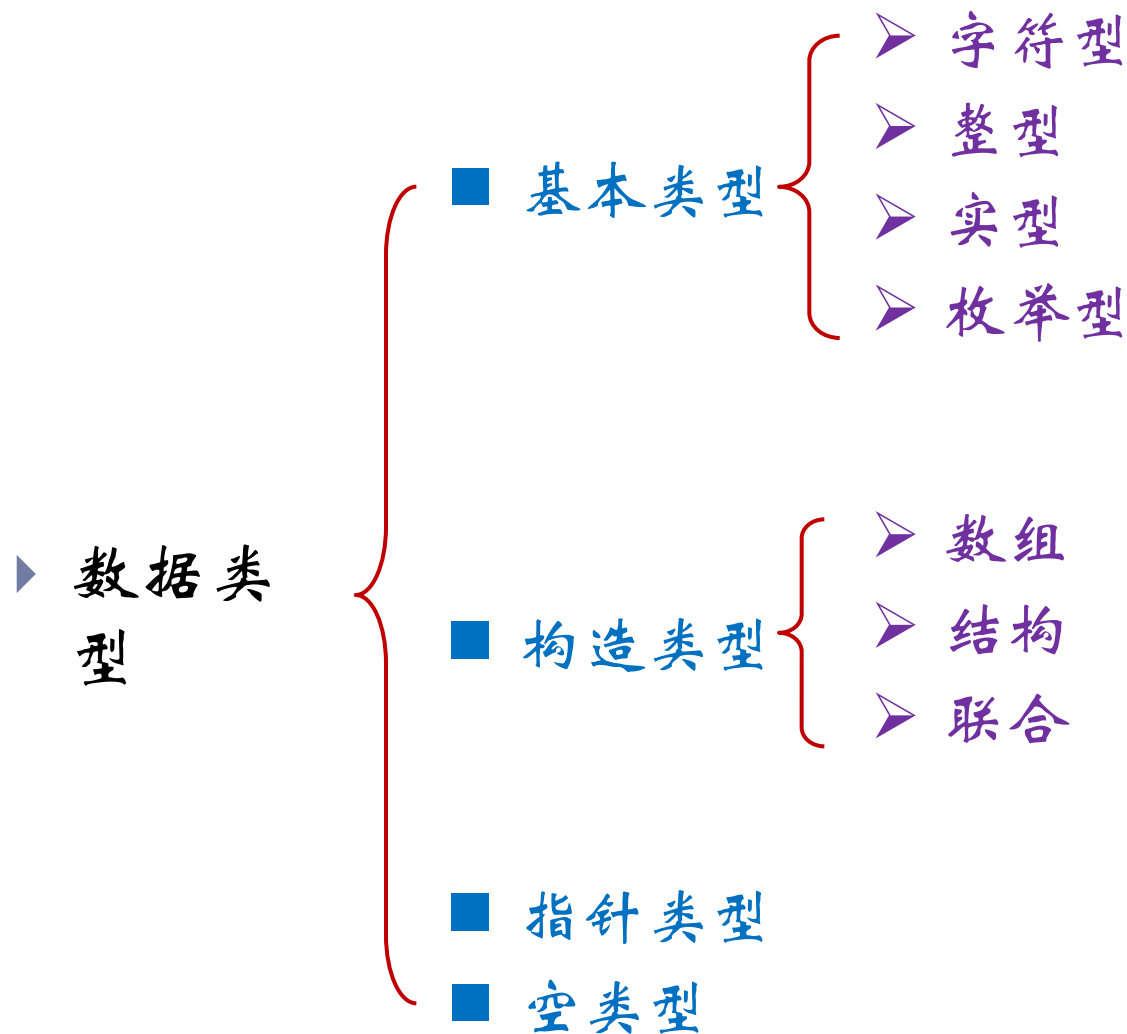
```
extern int Conter;
```

```
void strcpy(const char *Tar, char *Src);
```

▶ 变量的申明通常是给编译器以指示，并不实际分配内存。
因此通常放在头文件中。



C语言的数据类型





C语言基本数据类型

- ▶ C语言的基本数据类型包含：字符型、整型、实型、枚举型
- ▶ 不同的C编译器可能对一些类型的在内存中的字节数规定不同，可以用sizeof()检测各种类型所需的存储字节数。

C语言基本数据类型	字节数
unsigned/signed char	1
unsigned/signed short (int)	2
unsigned/signed int	4
unsigned/signed long (int)	4
unsigned/signed float	
enum	4
unsigned/signed int64	8
unsigned/signed double	

Windows API类型	存储字节数
BYTE	1
WORD	2
DWORD	4



数组

▶ 数组的定义

```
int Array[5]={1,2,3,4,5};
```

存储类型 数据类型 数组名[元素个数]={初始值表};

▶ 数组的初值

▶ 数组在内存中的存储：顺序存储

▶ 数组的引用：以元素下标引用

▶ 下标总是从0开始，到元素个数-1

▶ 数组引用的实质：地址+偏移

▶ 数组等同于一个指针

▶ 数组的申明

```
extern int Array[3];
```

Array

0

1

1

2

2

3

3

4

4

5



字符数组与字符串

▶ 字符数组

- ▶ 定义：跟上面的数组一样，数据类型为char
- ▶ 初值：跟上面一样，但增加了一种

```
char str[] = "Hello";  
char str[] = {'H', 'e', 'l', 'l', 'o', 0};
```

▶ 字符串：字符数组的字符串赋值形式，自动加0的字符数组

```
char *str = "Hello";
```



函数

▶ 函数定义

存储类型 返回类型 函数名(形参列表) {函数体语句}

返回类型: void型相当于子程序

形式参数: 传值型、传址型。注意申明形式

函数调用: 非void返回值可赋给变量;可直接作语句用。

```
int max(int x, int y)
{
}
```

▶ 函数申明

extern 返回类型 函数名(形参[类型]列表);

注意: 有; 符号。形参可只给类型。

通常函数申明放在头文件中。有时也叫函数原型。

```
int max(int x, int y);
extern int min(int x,int y);
```



程序结构

- ▶ 顺序结构
 - ▶ 按照语句的顺序执行
- ▶ 流程控制
 - ▶ 选择结构
 - ▶ if – else
 - ▶ switch
 - ▶ 循环结构
 - ▶ for
 - ▶ while { }
 - ▶ do { } while
 - ▶ do { } until
 - ▶ 跳转 goto

□ 1966年，Bohra和Jacopini证明了可以用三种结构表达以上各种结构

- 顺序结构
- 选择结构 if – else
- 循环结构 for



结构

▶ 结构定义

```
struct 结构名 {  
    结构成员类型    结构成员名;  
    .....  
};
```

▶ 结构使用

```
struct 结构名 变量名;
```

```
struct StudentInfo {  
    int StID;  
    char Name[12];  
};
```

```
struct StudentInfo aStudent;  
struct StudentInfo Students[35];  
struct StudentInfo *lpStuentsList;  
int x = aStudent.StID;  
int y = Students[10].StID;
```



联合

▶ 联合定义

union 联合名 {

联合成员类型 联合成员名;

.....

};

▶ 联合的使用

union 联合名 变量名;

```
union Person aPerson;  
union Person aPerson[35];  
int StId = aPerson.St.StID;  
int ResId = aPerson.Res.CertID;
```

▶ 结构对齐

▶ 注意：不同的编译器使用不同的结构对齐策略，如字节对齐、字对齐、双字对齐

▶ 使用编译申明：

▶ #pragma pack(1) 迫使进行字节对齐



联合-示例

```
struct Resident {      | struct Student {
    int    CertID;      |     int    StID;
    char   Name[16];    |     char   Name[12];
};                      |     };

union Person {          |
    struct Resident      |     Res;
    struct Student      |     St;
    Char               |     InfoData[20];
};
```




自定义数据类型

- ▶ 使用typedef定义自定义的数据类型

typedef 原数据类型

用户数据类型

- ▶ 使用：等同于系统预定义的数据类型
- ▶ 自定义数据类型与宏定义不同，宏定义只是展开，因此数据类型不变，而自定义类型则不同，是一种新的数据类型。很多编译器在编译时进行类型检查，以确定是否违例

```
typedef unsigned char BYTE
```

```
typedef struct tagStudent {  
    int StID;  
    char Name[12];  
} Student, FAR *LpStudent;
```

```
BYTE    aByte, *lpByte;  
Student aStudent;  
LpStudent lpStudentList;
```

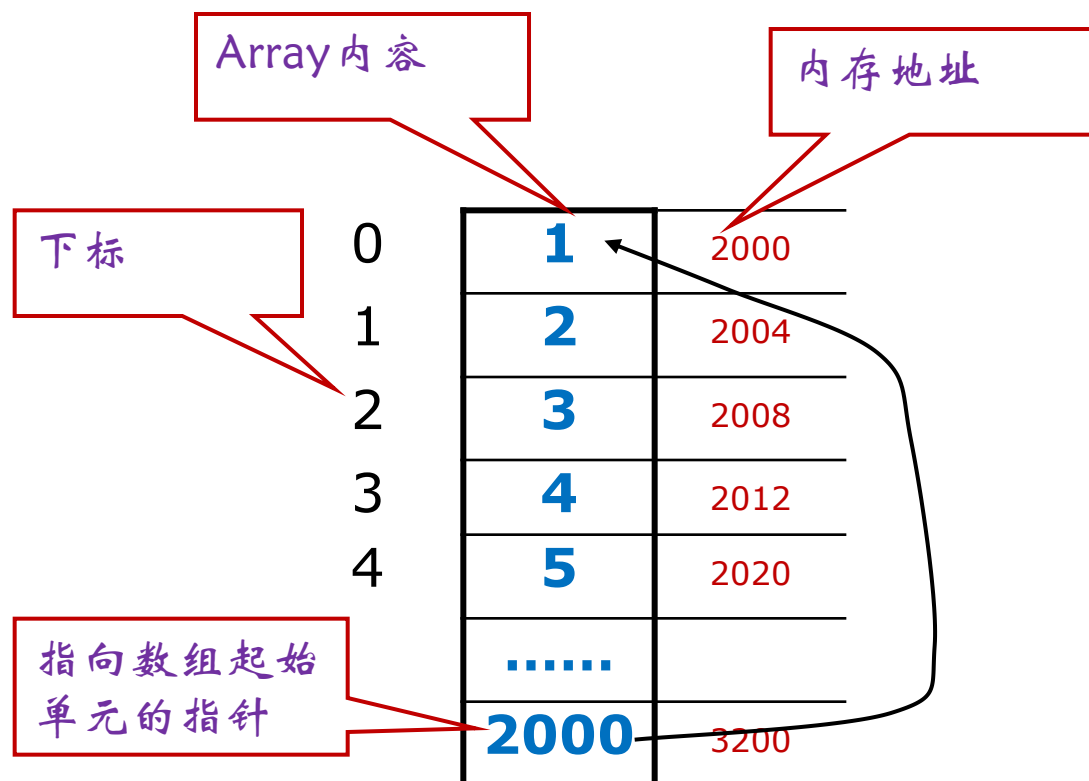


地址指针

- ▶ 程序在内存中才能得以执行，数据在内存中才能被直接访问。内存单元是以地址标记的。
- ▶ 定义：
 - ▶ 数据类型 * 变量名;
 - ▶ `int *MyInt;`
 - ▶ `struct Student *lpStudentsList;`
- ▶ 使用：
 - ▶ `*MyInt = 30;`
 - ▶ `lpStudentsList->StID = 1234;`
- ▶ 注意：指针是一种类型，指针变量也占用内存。



地址指针-详细说明





函数指针

- ▶ 每一个函数被编译成代码后，都会装入内存以执行，它们在内存中都会有一个入口地址。可以用一个变量表示这个入口地址，称为指向这个函数的指针。
- 在编写程序时，程序还没编译，如何知道函数指针呢？
 - 首先申明一个与该函数匹配的函数指针类型；
 - 定义一个该类型的变量；
 - 把一个函数的指针赋给这个变量。



函数指针的使用

```
int max(int x, int y) {...}
```

```
int (*FunPtr)(int, int);  
FunPtr = max;
```

```
typedef int (*MaxFun)(int, int);  
MaxFun MaxFunPtr;  
MaxFunPtr = max;
```

```
int MaxVal = MaxFunPtr(3, 4);
```



使用指针时的注意事项

- ▶ 可以定义指针类型的变量，但使用该指针变量前，必须初始化，否则不知道该指针变量指向何处，造成结果混乱，甚至造成访问违例(Exception)。
- ▶ 空指针用NULL表示。指针变量的值为空(NULL)，实际上是0。
- ▶ 指针变量的值存放的是地址，可以进行+,-,++,--运算。
- ▶ 指针分为短指针(32位)和长指针(64位)。
- ▶ 指针变量可以作为函数的参数进行传递。Windows API大量使用回调函数，就是将函数作为参数的。
- ▶ 一个指针变量可指向另一个指针变量的地址,即指针的指针
- ▶ void指针类型：首先表明是一个指针，但具体未指明是那一种类型的指针。



编程最痛苦的事

世界上最遥远的距离不是生与死，而是你亲手制造的Bug就在你眼前，你却怎么都找不到她。



“我爱你”三个字，讲出来只要三秒钟，解释要三小时，证明却要一辈子。“bug”三个字母，发现需要三秒，找到需要三小时，debug却要一辈子。这真的是多么痛彻的领悟。



小结

► C语言的难点

- 变量的可用范围容易出错
- 指针的指向易出错，特别是多级指针
- 运算符的优先顺序易出错
- 结构体使用时要注意其字节对齐的问题（1字节对齐或4字节对齐），多位开发人员不一致则会出错



1.4 抽象数据类型

- ▶ 数据类型：Data Type — 高级语言中指一组性质相同的**值集合**及其上可进行的**操作**的总称。

C语言提供int, char, float, double等基本数据类型；数组、结构体、共用体、枚举等构造数据类型；
还有指针、空(void)类型等。
用户也可用typedef 定义新的数据类型。

此类型可以分为**原子类型**和**结构类型**。
原子类型是不可分的数据类型。
结构类型是可分为原子类型的数据类型。

为提高复用性？我们可以对不同的数据结构进行抽象，形成新的数据类型吗？



1.4 抽象数据类型

- ▶ 抽象数据类型Abstract Data Type — 是指一个数学模型以及定义在该模型上的一组操作。抽象数据类型的定义取决于它的一组**逻辑特性**，而与其在**计算机内部的表示和实现无关**。
- ▶ 一个含抽象数据类型的程序模块通常包含**定义**、**表示**和**实现**三个部分。
- ▶ 按抽象数据类型的数据取值的特性将其分为三种
 - ▶ 原子类型atomic data type
 - ▶ 固定聚结类型fixed-aggregate data type
 - ▶ 可变聚结类型variable-aggregate data type
- ▶ 多形数据类型polymorphic data type — 是指其值的成分不确定的数据类型。



1.4 抽象数据类型

- 抽象数据类型可用 (D, S, P) 三元组表示
其中， D 是数据对象， S 是 D 上的关系集， P 是对 D 的基本操作集。

ADT 抽象数据类型名 {

 数据对象：〈数据对象的定义〉

 数据关系：〈数据关系的定义〉

 基本操作：〈基本操作的定义〉

} ADT 抽象数据类型名



1.4 抽象数据类型

数据对象、数据关系用伪码描述；基本操作定义格式为

基本操作名（参数表）

初始条件：〈初始条件描述〉

操作结果：〈操作结果描述〉

- ▶ 基本操作有两种参数：赋值参数只为操作提供输入值；引用参数以&打头，除可提供输入值外，还将返回操作结果。
- ▶ “初始条件”描述了操作执行之前数据结构和参数应满足的条件，若不满足，则操作失败，并返回相应出错信息。
- ▶ “操作结果”说明了操作正常完成之后，数据结构的变化状况和应返回的结果。若初始条件为空，则省略之。



1.4 抽象数据类型

抽象类型三元组定义

```
ADT Triplet {  
    数据对象:  $D = \{e1, e2, e3 | e1, e2, e3 \in \text{ElemSet} \text{ (定义了关系运算的某个集合)}\}$   
    数据关系:  $R1 = \{ \langle e1, e2 \rangle, \langle e2, e3 \rangle \}$   
    基本操作:  
        InitTriplet( &T, v1, v2, v3 )  
            操作结果: 构造了三元组 T, 元素 e1, e2 和 e3 分别被赋以参数 v1, v2 和 v3 的值。  
        DestroyTriplet( &T )  
            操作结果: 三元组 T 被销毁。  
        Get( T, i, &e )  
            初始条件: 三元组 T 已存在,  $1 \leq i \leq 3$ 。  
            操作结果: 用 e 返回 T 的第 i 元的值。  
        Put( &T, i, e )  
            初始条件: 三元组 T 已存在,  $1 \leq i \leq 3$ 。  
            操作结果: 改变 T 的第 i 元的值为 e。  
        IsAscending( T )  
            初始条件: 三元组 T 已存在。  
            操作结果: 如果 T 的 3 个元素按升序排列, 则返回 1, 否则返回 0。  
        IsDescending( T )  
            初始条件: 三元组 T 已存在。  
            操作结果: 如果 T 的 3 个元素按降序排列, 则返回 1, 否则返回 0。  
        Max( T, &e )  
            初始条件: 三元组 T 已存在。  
            操作结果: 用 e 返回 T 的 3 个元素中的最大值。  
        Min( T, &e )  
            初始条件: 三元组 T 已存在。  
            操作结果: 用 e 返回 T 的 3 个元素中的最小值。  
}ADT Triplet
```



1.4 抽象数据类型

抽象类型三元组实现

```
// ----- 采用动态分配的顺序存储结构 -----  
typedef ElemType *Triplet; // 由 InitTriplet 分配 3 个元素存储空间
```

```
// ----- 基本操作的函数原型说明 -----  
Status InitTriplet (Triplet &T, ElemType v1, ElemType v2, ElemType v3);  
// 操作结果:构造了三元组 T,元素 e1,e2 和 e3 分别被赋以参数 v1,v2 和 v3 的值。  
Status DestroyTriplet (Triplet &T);  
// 操作结果:三元组 T 被销毁。  
Status Get (Triplet T, int i, ElemType &e);  
// 初始条件:三元组 T 已存在,1≤i≤3。  
// 操作结果:用 e 返回 T 的第 i 元的值。  
Status Put (Triplet &T, int i, ElemType e);  
// 初始条件:三元组 T 已存在,1≤i≤3。  
// 操作结果:改变 T 的第 i 元的值为 e。  
Status IsAscending (Triplet T);  
// 初始条件:三元组 T 已存在。  
// 操作结果:如果 T 的 3 个元素按升序排列,则返回 1,否则返回 0。  
Status IsDescending (Triplet T);  
// 初始条件:三元组 T 已存在。  
// 操作结果:如果 T 的 3 个元素按降序排列,则返回 1,否则返回 0。  
Status Max (Triplet T, ElemType &e);  
// 初始条件:三元组 T 已存在。  
// 操作结果:用 e 返回 T 的 3 个元素中的最大值。  
Status Min (Triplet T, ElemType &e);  
// 初始条件:三元组 T 已存在。  
// 操作结果:用 e 返回 T 的 3 个元素中的最小值。
```

```
// ----- 基本操作的实现 -----  
Status InitTriplet (Triplet &T, ElemType v1, ElemType v2, ElemType v3) {  
    // 构造三元组 T,依次置 T 的 3 个元素的初值为 v1,v2 和 v3。  
    T = (ElemType *) malloc (3 * sizeof(ElemType)); // 分配 3 个元素的存储空间  
    if (!T) exit(OVERFLOW); // 分配存储空间失败  
    T[0] = v1;    T[1] = v2;    T[2] = v3;  
    return OK;  
}  
// InitTriplet  
Status DestroyTriplet (Triplet &T) {  
    // 销毁三元组 T。  
    free(T);    T = NULL;  
    return OK;  
}  
// DestroyTriplet  
Status Get (Triplet T, int i, ElemType &e) {  
    // 1≤i≤3,用 e 返回 T 的第 i 元的值。  
    if (i<1 || i>3) return ERROR;  
    e = T[i-1];  
    return OK;  
}  
// Get  
Status Put (Triplet &T, int i, ElemType e) {  
    // 1≤i≤3,置 T 的第 i 元的值为 e。  
    if (i<1 || i>3) return ERROR;  
    T[i-1] = e;  
    return OK;  
}
```

```
// Put  
Status IsAscending (Triplet T) {  
    // 如果 T 的 3 个元素按升序排列,则返回 1,否则返回 0。  
    return (T[0] <= T[1]) && (T[1] <= T[2]);  
}  
// IsAscending  
Status IsDescending (Triplet T) {  
    // 如果 T 的 3 个元素按降序排列,则返回 1,否则返回 0。  
    return (T[0] >= T[1]) && (T[1] >= T[2]);  
}  
// IsDescending  
Status Max (Triplet T, ElemType &e) {  
    // 用 e 返回指向 T 的最大元素的值。  
    e = (T[0] >= T[1]) ? ((T[0] >= T[2]) ? T[0] : T[2])  
        : ((T[1] >= T[2]) ? T[1] : T[2]);  
    return OK;  
}  
// Max  
Status Min (Triplet T, ElemType &e) {  
    // 用 e 返回指向 T 的最小元素的值。  
    e = (T[0] <= T[1]) ? ((T[0] <= T[2]) ? T[0] : T[2])  
        : ((T[1] <= T[2]) ? T[1] : T[2]);  
    return OK;  
}  
// Min
```



小结

- ▶ 抽象数据类型为了提高软件模块的复用程度
 - 其定义与仅取决于逻辑特性，具有高度抽象性
 - 其表示和实现与实际的存储结构关联
 - 多形数据类型更适合使用C++等面向对象的语言实现



1.5 算法和算法分析

- ▶ 算法(algorithm)—解决某一特定问题的具体**步骤和方法**的描述。是指令的有限序列。
- ▶ 算法特性——
 - ▶ 有穷性 — 一个算法必须在执行有限步骤之后结束
 - ▶ 确定性 — 算法的每一步必须是确切定义的，
不能产生二义性
 - ▶ 可行性 — 算法是能行的，通过有限次运行实现，能够
得到确定的结果
 - ▶ 输出 — 一个算法有零个或多个输出
 - ▶ 输入 — 一个算法有零个或多个输入



算法描述

- ▶ 算法的描述方法：
 - ▶ 用自然语言描述
 - ▶ 用流程图描述
 - ▶ 用伪代码描述
 - ▶ 用计算机高级语言描述。
- ▶ 本书采用C语言描述算法。

数据结构 + 算法 = 程序设计

问题的数学模型

处理问题的策略

为计算机处理问题编制一组指令集



算法评价

- ▶ 算法的评价 — 衡量算法优劣的标准,也是算法设计的要求
 - ▶ 正确性(correctness):满足具体问题求解要求
 - ▶ 健壮性(robustness):在非法数据输入时仍能处理
 - ▶ 可读性(readability):易于阅读理解、调试和修改
 - ▶ 资源需求与效率:低需求、高效率。一个算法为完成求解特定问题,需要使用一些资源,其中最主要的CPU和内存。通常又称为算法的复杂性,分为时间复杂性(对CPU运算时间的需求)和空间复杂性(对内存资源的需求)。



算法复杂性

- ▶ 一个算法实现需要资源。特别是CPU和内存资源。通常又称为**算法的复杂性**，分为**时间复杂性**(对CPU运算时间的需求)和**空间复杂性**(对内存资源的需求)。
- ▶ 时间和空间都与问题的规模有关。
- ▶ 例如：排序问题

```
void bubble_sort ( int a[], int n) {  
    //将a中的整数序列重新排列成自小至大有序的整数序列  
    for (int i = n-1, change = TRUE; i > 1 && change; i--){  
        change = FALSE;  
        for (j=0; j<i; j++)  
            if( a[j]>a[j+1]){swap(a[j],a[j+1]);change = TRUE;}  
    }  
} //bubble_sort
```

冒泡排序的算法时间复杂度分析:

$$T(n) = \sum_{i=1}^{n-1} (n - i) = \frac{n(n-1)}{2}$$



算法效率

- ▶ 算法的时间复杂性即算法效率——通常以用该算法编制的程序在计算机上执行所消耗的时间来度量。
- ▶ 事后统计——利用计算机内记时功能,不同算法的程序可以用一组或多组相同的统计数据区分;

算法的某些部位插入时间函数time()

- ▶ 事前分析估计——一个高级语言程序在计算机上运行所消耗的时间取决于:

- ① 依据的算法选用何种策略
- ② 问题的规模
- ③ 程序语言
- ④ 编译程序产生机器代码质量
- ⑤ 机器执行指令速度



算法效率

- ▶ 然而，同一个算法用不同的语言、不同的编译程序、在不同的计算机上运行，效率均不同，所以使用**绝对时间单位**衡量算法效率**不合适**
- ▶ 因此，我们通常用事先估计方法，选择算法所需的基本操作次数来衡量算法的时间复杂性。例如：比较次数、移动数据单元的次數等。
- ▶ 时间复杂度（**渐进的时间复杂度**）：基本操作重复执行的次数的阶数 **(大O记号): $T(n) = O(f(n))$**

- 一个算法的“运行时间”通常是随问题规模的增长而增长，它是问题规模(用正整数 n 表示)的函数。
- 统计算法中的**关键操作**，以**关键操作重复执行的次数**作为算法的时间度量。
- 算法中关键操作重复执行的次数是规模 n 的某个函数 $T(n)$ ，称 $T(n)$ 为算法的(渐近)时间复杂度。



渐进时间复杂度

► 数量级的思想

例 1：求两个方阵的乘积 $C=A*B$

MATRIXMLT(float A[n][n],float B[n][n],float C[n][n])

```
{ int i,j,k;
  for(i=0;i<n;i++)           //n+1
    for(j=0;j<n;j++)         //n(n+1)
      { C[i][j]=0;           //n*n
        for( k=0;k<n;k++)     //n*n*(n+1)
          C[i][j]+=A[i][k]*B[k][j]  //n*n*n
        }
      }
```

$$T(n) = 2n^3 + 3n^2 + 2n + 1 = O(n^3)$$



大O表示的渐进时间复杂度

- ▶ 通常用 $O(1)$ 表示常数计算时间。常见的渐进时间复杂度有：

$$O(1) < O(\log_2 n) < O(n) < O(n \log_2 n) < O(n^2) < O(n^3) < O(2^n) < O(3^n) < O(n!)$$

- ▶ $T(n) = O(f(n))$ 的几条规则：

- **加法规则** 针对并列程序段

$$\begin{aligned} T(n, m) &= T1(n) + T2(m) \\ &= O(\max(f(n), g(m))) \end{aligned}$$

- **乘法规则** 针对嵌套程序段

$$\begin{aligned} T(n, m) &= T1(n) * T2(m) \\ &= O(f(n) * g(m)) \end{aligned}$$

- **乘法规则中的常数无关项** $O(C) \rightarrow O(1)$

$$T(n) = O(c * f(n)) = O(f(n))$$

- **$O(1)$ 表示常数计算时间**



空间复杂度

- ▶ 空间复杂度:

$$s(n)=O(f(n))$$

n 为问题的规模(或大小)。

- ▶ 空间复杂性通常考虑排除装载问题本身所必需的数据空间和算法所必需的空间外，额外所需的空間。
- ▶ 若额外空间相对于输入数据量来说是常数，则称此算法为原地工作。



小结

- ▶ 从逻辑上，算法是解决问题的一系列步骤和方法
- ▶ 从程序上，算法是有限指令集
- ▶ 要从时间、空间两个方面评估算法的优劣，包括渐进时间复杂度和空间复杂度



Take-home Message

- 计算思维对于人工智能非常重要
- 数据结构的基本概念及相关名词术语
 - 数据结构包括数据的逻辑结构、存储结构及操作算法三部分。
 - 数据的逻辑结构分为线性结构和非线性结构两种。
 - 数据的存储结构分为顺序存储和链接存储两种。
 - 名词术语包括数据、数据元素、数据对象、数据类型、抽象数据类型等。
- 回顾C语言的语法和基本内容
- 理解算法的特性及算法的评价标准，熟悉算法的时间复杂度和空间复杂度的分析方法。